Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Departamentul Inginerie Software și Automatică

Lucrare de laborator nr. 1

Profesor: Torică Eugeniu
Student: Zlatovcen Bogdan, grupa TI-251M

Chișinău, 2026

1 Rezultatele aplicației secvențiale

Prima etapă a constat în compilarea și rularea aplicației secvențiale. Aceasta verifică nonce-urile unul câte unul, până când găsește prima valoare care satisface condiția impusă asupra hash-ului.

Tabela 1: Timpii obținuți pentru aplicația secvențială

n	Soluție găsită	Timp execuție
5	`Hello World616577`	197 ms
6	`Hello World1439621`	462 ms
7	`Hello World309680830`	102 838 ms

Hash-urile verificate pentru soluțiile obținute sunt prezentate în tabelul următor. Fiecare valoare începe cu numărul de zerouri cerut de dificultatea n.

Tabela 2: Hash-uri SHA-256 verificate

n	Hash SHA-256
5	`00000efa601fff0c6b8c79c6a95031c69aa6f6f821c214316f26c6051a5275e7`
6	`0000008a53e365f8f4145f2cf4b502b29949b24746484ef2b556aaa5e2d7a562`
7	`00000002faa68ff798cd723e66b4f8f04abae180d5e52d95dde21cfb8781b15f`

Se observă o creștere puternică a timpului atunci când dificultatea crește. Pentru $n = 7$, timpul ajunge la aproximativ 1 minut și 43 de secunde. Acest comportament este așteptat, deoarece SHA-256 produce valori distribuite pseudoaleator, iar șansa de a obține un prefix cu mai multe zerouri scade exponențial.

2 Rezultatele aplicației OpenMP

Varianta OpenMP folosește paralelism cu memorie partajată. Ideea principală este împărțirea spațiului de căutare între mai multe fire de execuție. Fiecare fir poate calcula hash-uri independent, iar sincronizarea este necesară doar pentru oprirea căutării și raportarea soluției găsite.

Pentru $n = 6$, timpul de execuție în funcție de numărul de fire este prezentat în tabelul următor.

Tabela 3: Timpul de execuție OpenMP pentru $n = 6$

Număr fire	1	2	4	8	16	32
Timp execuție, ms	473	242	124	66	49	29
Accelerare	1.00	1.95	3.81	7.17	9.65	16.31
Eficiență	1.00	0.98	0.95	0.90	0.60	0.51

Se observă o scalare foarte bună până la 8 fire. După acest punct, timpul continuă să scadă, dar eficiența scade mai vizibil. Aceasta indică apariția costurilor de sincronizare, a competiției pentru resursele procesorului și a efectelor de suprasubscriere.

Pentru comparația cu aplicația secvențială au fost rulate aceleași valori ale dificultății folosind 4 fire OpenMP.

Pentru $n = 6$, trecerea de la rularea secvențială de 462 ms la rularea OpenMP cu 32 de fire de 29 ms oferă o accelerare de aproximativ:

$$S = \frac{462}{29} \approx 15.9.$$

Tabela 4: OpenMP cu 4 fire pentru dificultăți diferite

n	Timp execuție
5	64 ms
6	122 ms
7	29 229 ms

3 Rezultatele aplicației OpenMPI

Varianta OpenMPI folosește procese separate, fiecare cu memorie proprie. Comunicarea dintre procese se face explicit, prin mecanisme MPI. Acest model este util pentru sisteme distribuite și clustere, însă pe o singură mașină locală poate introduce overhead mai mare decât OpenMP.

Pentru $n = 6$, timpul de execuție în funcție de numărul de procese este prezentat în tabelul următor.

Tabela 5: Timpul de execuție OpenMPI pentru $n = 6$

Număr procese	1	2	4	8	16	32
Timp execuție, ms	556	379	354	519	195	188
Accelerare	1.00	1.47	1.57	1.07	2.85	2.96
Eficiență	1.00	0.73	0.39	0.13	0.18	0.09

Rezultatele OpenMPI sunt mai puțin uniforme decât cele OpenMP. Pentru 8 procese timpul este mai mare decât pentru 4 procese, ceea ce arată influența overhead-ului de comunicare și de coordonare. La 16 și 32 procese se observă o îmbunătățire, dar eficiența rămâne scăzută, deoarece procesele concurează pentru resursele aceleiași mașini locale.

Au fost rulate și valorile $n = 5$, $n = 6$, $n = 7$, folosind 4 procese MPI.

Tabela 6: OpenMPI cu 4 procese pentru dificultăți diferite

n	Timp execuție
5	96 ms
6	354 ms
7	35 110 ms

Pentru $n = 6$, procesele au identificat aceeași soluție: `Hello World1439621`. Acest lucru confirmă că împărțirea spațiului de căutare și comunicarea dintre procese au fost implementate corect.

4 Utilitatea librăriilor și analiza experienței practice

OpenMP este potrivit pentru aplicații care rulează pe un singur calculator sau pe un singur nod și care au bucle independente, intensive din punct de vedere computațional. Avantajul său este simplitatea: programatorul poate paraleliza anumite regiuni ale codului fără a construi manual un sistem de comunicare între procese. Limitarea principală este faptul că OpenMP folosește memorie partajată și, în mod obișnuit, nu scalează direct pe mai multe noduri fizice.

OpenMPI este potrivit pentru aplicații distribuite, unde calculul poate fi împărțit între procese care rulează pe noduri diferite. Acest model oferă scalabilitate mai bună pentru clustere, dar cere comunicare explicită, sincronizare atentă și gestionarea costurilor de transfer de mesaje.

În contextul experienței mele de dezvoltare fullstack, nu am avut până acum o problemă prac-

tică în care să fie necesară paralelizarea explicită a unui algoritm cu OpenMP sau OpenMPI. În aplicațiile web obișnuite, paralelismul există, dar este de obicei gestionat de framework-uri, servere web, baze de date, containere sau servicii cloud, nu direct de programator la nivel de fire de execuție sau procese MPI.

Un exemplu relevant pentru paralelism este procesarea video. Fiecare fragment video poate fi prelucrat în paralel, dar încărcarea, validarea, combinarea și publicarea rezultatului rămân etape secvențiale. Partea secvențială este reprezentată de pașii care trebuie executați într-o anumită ordine: încărcarea fișierului, verificarea fișierului, extragerea metadatelor, împărțirea videoclipului în fragmente, combinarea rezultatelor și publicarea finală. Partea paralelă apare atunci când videoclipul este împărțit în mai multe bucăți, iar fiecare bucată poate fi procesată simultan de fire de execuție sau de procese diferite. De exemplu, unele fragmente pot fi convertite în 720p, altele în 1080p, iar alte operații pot rula simultan pe alte nuclee sau pe alte servere.

Crypto-puzzle-ul din laborator are un grad ridicat de paralelism, deoarece verificarea unui nonce este independentă de verificarea altui nonce. Totuși, există și părți secvențiale: inițializarea programului, pornirea firelor sau proceselor, comunicarea stării de oprire și afișarea rezultatului final. Din acest motiv, accelerarea reală nu poate crește nelimitat.

5 Interpretarea rezultatelor prin legea lui Amdahl

Legea lui Amdahl descrie accelerarea maximă obținută prin paralelizare:

$$S(N) = \frac{1}{(1-p) + \frac{p}{N}},$$

unde $S(N)$ este accelerarea pentru N unități de execuție, iar p este fracția paralelizabilă a programului.

Conform acestei legi, chiar dacă o mare parte a programului poate fi paralelizată, partea secvențială limitează accelerarea finală. În plus, implementările reale includ overhead suplimentar: crearea firelor sau proceselor, sincronizarea, accesul la memorie, comunicarea și oprirea coordonată după găsirea soluției.

În cazul OpenMP, accelerarea pentru $n = 6$ este foarte bună până la 8 fire. Pentru 32 de fire, timpul scade la 29 ms, ceea ce reprezintă aproximativ $15.9\times$ față de rularea secvențială și $16.31\times$ față de varianta OpenMP cu un singur fir. Totuși, accelerarea este sub limita ideală de $32\times$, deoarece partea secvențială și overhead-ul nu dispar.

În cazul OpenMPI, accelerarea este mai modestă. La 32 de procese se obține doar aproximativ $2.96\times$ față de rularea MPI cu un proces. Această diferență apare deoarece MPI folosește procese separate și comunicare explicită, iar testele au fost executate local, nu pe un cluster cu noduri fizice separate. Pentru această problemă și pentru acest mediu de rulare, OpenMP este mai eficient decât OpenMPI.

6 MD5 pentru parole și practici moderne

În trecut, pentru păstrarea parolelor se foloseau uneori funcții hash rapide, precum MD5. Această practică este considerată nesigură. Problema principală este că MD5 este foarte rapid, iar acest lucru permite unui atacator să testeze un număr foarte mare de parole candidate într-un timp scurt, mai ales folosind GPU-uri sau baze de date pregenerate.

Atacul tipic apare după compromiterea unei baze de date care conține hash-uri ale parolelor. Dacă parolele sunt stocate ca MD5 simplu, atacatorul poate aplica brute-force, dicționare de parole comune sau rainbow tables pentru a găsi rapid parolele originale, mai ales dacă utilizatorii au parole slabe sau reutilizate.

Practica modernă este folosirea unor funcții specializate pentru parole, lente și parametrizabile, precum bcrypt, scrypt sau Argon2. Acestea trebuie folosite împreună cu un salt unic pentru fiecare utilizator. În sisteme mai sensibile se poate folosi și un pepper păstrat separat de baza de date. De asemenea, costul algoritmului trebuie ajustat periodic, astfel încât verificarea unei parole să fie acceptabilă pentru aplicație, dar costisitoare pentru atacuri masive offline.

Crypto-puzzle-ul din acest laborator seamănă mai mult cu proof-of-work decât cu stocarea parolelor. În proof-of-work, rezolvarea trebuie să fie dificilă, iar verificarea rapidă. Pentru parole, scopul este diferit: reducerea eficienței atacurilor offline prin funcții de derivare lente, cu salt și cost configurabil.

7 Concluzii

Laboratorul a demonstrat diferența practică dintre paralelizarea cu OpenMP și paralelizarea cu OpenMPI. Pentru problema testată, verificarea nonce-urilor este aproape independentă, ceea ce face ca aplicația să fie potrivită pentru paralelizare.

OpenMP a oferit cele mai bune rezultate în mediul local: pentru $n = 6$, timpul a scăzut până la 29 ms cu 32 de fire. OpenMPI a funcționat corect, dar a avut overhead mai mare, iar cea mai bună valoare pentru $n = 6$ a fost 188 ms la 32 de procese.

Rezultatele confirmă legea lui Amdahl: accelerarea depinde nu doar de numărul de fire sau procese, ci și de partea secvențială și de costurile de coordonare. Pentru o singură mașină locală, OpenMP este mai eficient în acest laborator. OpenMPI ar deveni mai util într-un context distribuit real, unde procesele ar rula pe mai multe noduri și ar avea resurse fizice separate.